

TP4

Edge Detection

The objective of this practical work is to implement an edge detection approach using gradients and the Canny edge detector.

Exercise 1 : Gradient computation and display

We are going to write a program that opens a PGM image, smooths it, computes the image gradients (I_x, I_y) by computing the derivatives in x and y using the Scharr operators. Then it computes the magnitude and angle of the gradient vector with respect to the x-axis, and finally displays the magnitude of the gradients by mapping them into the image viewing range $[0, 255]$. Check the file *displayGradientMagnitudes.c*

- Reuse the functions from the previous TP to smooth the images: copy the *filters.c filters.h* into the Utils folder.
- Implement the functions in *TP4/gradients.c*. Note: you can reuse your convolution functions from TP3 and use the Scharr filters to compute the derivatives.
- Test the program by running

```
% ./displayGradientMagnitudes images/boat.pgm  
results/boat_gradient_5_1.pgm 5 1
```

- Use it on different images (*coins.pgm*, *dog.pgm*, *moon.pgm*) and explore the effect of the initial smoothing operation.

Exercise 2: Canny's Edge Detection

We are now going to implement the Canny's approach¹. Check the file *cannyEdges.c*. It is composed of several steps:

1. Noise reduction by smoothing the image
2. Gradient calculation (e.g. using Sharr)
3. The magnitude and angle of the gradient are computed
4. Non maximum suppression using the angle and the magnitude
5. Hysteresis threshold using a double threshold (hight and low)

The first three items are already implemented with your previous work. In *TP4/canny.c* you have two prototypes for the missing functions: non-maximum suppression and Hysteresis threshold.

- Implement the functions in *TP4/canny.c*
- Test the program by running

```
% ./cannyEdges images/boat.pgm
                      results/boat_canny_5_1_50_10.pgm 5 1 50 10
```

- Explore other values for the high and low thresholds and the smoothing.
- Use it on different images (*coins.pgm*, *dog.pgm*, *moon.pgm*) and explore the effect of the high and low thresholds.

Exercise 3 (Bonus): display angles

Create a program *displayGradientAngles* similar to *displayGradientMagnitudes* that displays the angles of the detected gradients. For that purpose, create a function that converts angle orientation into colors and store them in a .ppm file, e.g.

```
% ./displayGradientAngles images/boat.pgm
                      results/boat_gradient_angles_5_1.ppm 5 1
```

Consider the HSV colorspace, in which H is naturally defined by a 2D angle. Convert it to RGB to store the image.

¹see lecture document Image Features - Section 4.1.1